Regex Matching

```
>>> import re

>>> re.fullmatch(pattern, string)
```

regex matching function

regex pattern

string to be checked.

→ returns bool
→ True if pattern match, else False.

	Character / Pattern		Meaning	Example	
1	r'abc'		Exact match	'abc'	
2	.	r'a.c'	any single character	'rac' 'rnc'	'rbc' 'ryc'
3	*	r'a*'	0 or more of the preceding characters	'' (empty) 'aaa'	'a' 'aaaaaa'
4	+	r'a+'	1 or more of the preceding character	'a'	'aaa'
5	?	r'ab'	0 or 1 of the preceding character	'a'	'ab'
6	[...]	r'[abc]' r'[0-9]' r[A-Za-z]	character set → matches ~~one~~ 1 char that is either 1 of the inside [...]	'a' 'b' 'c' '0' '2' '8' 'A' 'B' 'C' 'd'	
7	[^...]	r'[^0-9]	Not one of these → ~~not~~ 1 char not of the inside [...]	'a' 'b' 'X' ~~'1'~~ ~~'2'~~	
8	{n} or {min, max}	r"[A-Za-z]{3}" r"[a-z]{2-5}"	Count repetitions — exact — a range — 2 to 5 letters	'aBX' ~~'AB'~~ ~~'a'~~	'cdy' 'abc' 'xyza'

	Pattern/usage		Meaning	Example
9	\d	r'\d'	→ any digit → same as [0-9]	'1' '2' '0'
10	\w	r'\w'	any letter, number or _ → [A-Za-z0-9_]	'a' 'A' '1' '_'
11	\s	r'\s'	Any white space → or tab → or newline	' ' ' '

You can combine various patterns and characters to form complex patterns.

More matches

>>> re.search(pattern, target)

checks if pattern appears anywhere in the target.

>>> re.match(patt, tar)

checks only from the start.

Matching special characters

→ use '\' ~~[illegible]~~

which is backslash.

eg. r'\.+.' → '.' or '....' or '.....'

r'a\+b' → 'a+b'

Grouping. using ( )

r'(abc)' → matches 'abc' as a single unit.

r'(Mr|Ms)' → matches Mr & Mas ✓

r'(ab)+' → matches repetitions → 'ab' or 'ababab'

r(Cat | Dog | Bat) → matches

→ 'Cat'
→ 'Dog'
→ 'Bat'

only ✓